

Electrónica Digital

Ingeniería Informática – FICH, UNL
Iván Peralta



Sistemas formales

Un **sistema formal** es un sistema abstracto compuesto por un **lenguaje formal**, **axiomas** y **reglas de inferencia** que se utilizan para construir un **modelo conceptual** de un objeto del mundo real.

Un sistema formal reduce el objeto estudiado **a símbolos que se expresan mediante fórmulas** que reflejan las relaciones entre los símbolos y pasar de una fórmula a otra.

Un sistema formal es como un juego con reglas muy estrictas. Al igual que el ajedrez: tiene piezas (símbolos), reglas sobre cómo mover cada pieza (reglas de inferencia) y una posición inicial (axiomas).



$$\text{DEPOSITAR}(B, N, M, B_)\equiv N \in \text{dom } B \wedge B_ = (B \setminus \{[N, S]\}) \cup \{[N, S+M]\}$$

Donde:

- B es el conjunto de pares ordenados [Número de cuenta, Saldo]
- N es el número de cuenta donde se realizará el depósito
- M es el monto a depositar ($M > 0$)
- S es el saldo actual de la cuenta N antes del depósito
- B_ es el conjunto resultante después de realizar el depósito

Esta especificación establece que:

- El número de cuenta N debe existir en el dominio de B
- El nuevo conjunto B_ se obtiene eliminando el par [N, S] del conjunto B y agregando el par [N, S+M], donde el saldo se incrementa en M

Los símbolos utilizados son:

- $\equiv$ (igual por definición)
- $\in$ (pertenece)
- $\wedge$ (y)
- $\setminus$ (diferencia de conjuntos)
- $\cup$ (unión)

Los sistemas formales se utilizan para:

- Modelar y analizar procesos computacionales

Ej. **Autómatas finitos**: Son modelos simples que ayudan a entender cómo se procesan secuencias de datos o caracteres, por ejemplo, al analizar **patrones** en textos.

- Desarrollar software

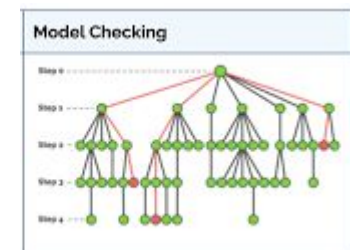
Ej. *Lenguajes de programación basados en formalismos* (como Haskell o **Prolog**): Estos lenguajes se fundamentan en reglas **matemáticas**, lo que permite escribir programas que, en teoría, tienen menos **errores**.

- Especificar y verificar los problemas

Ej. *Model checking*: Es una técnica que **examina** todas las **posibles** situaciones de un sistema para **asegurarse** de que cumpla con ciertas **propiedades** o reglas. Esto es fundamental en áreas como la **verificación** de software o hardware **crítico**, donde un error podría tener consecuencias importantes.



```
%prolog
| ?- consult(user).
| writeit :- write('Hola mundo'),nl.
| ^D user consulted, 10 msec 336 bytes
yes
| ?- writeit.
Hola mundo
yes
```



Las características clave de los métodos formales son:

- Utilizan **notaciones matemáticas** para representar la especificación, verificación y diseño de sistemas.
- Modelan y analizan procesos computacionales para **simularlos y estandarizarlos**, buscando metodologías para **garantizar la corrección** de las soluciones

Se aplican al desarrollo de diversos tipos de sistemas:

Sistemas críticos – son sistemas que se caracterizan por su **seguridad, confiabilidad y protección**, por lo que se debe **asegurar la confiabilidad** del software es esencial.

Ejemplos - sistemas de información de control de tráfico aéreo, sistemas de señalización de redes ferroviarias, sistemas de naves espaciales y sistemas de control médico.

En un controlador de tráfico aéreo, se usa un método formal para probar matemáticamente que dos aviones nunca tendrán rutas que los hagan colisionar.

Sistemas complejos - son sistemas en los cuales es difícil **probar propiedades deseadas** del sistema. Los métodos formales permiten especificar, verificar y validar sistemas manera más precisa, reduciendo la dependencia de la intuición y el juicio humano.

Ejemplos - sistemas de distribución de energía, agua y gas, sistemas de operación de una planta petroquímica.

En una red eléctrica, los métodos formales permiten modelar matemáticamente las interacciones entre generadores para garantizar que no ocurran apagones por sobrecarga.

Se aplican al desarrollo de diversos tipos de sistemas:

Sistemas heredados y de mantenimiento - Los métodos formales también se utilizan en el mantenimiento y evolución de sistemas heredados, donde es importante garantizar la corrección de los cambios realizados.

Un banco con software de los años 90 usa métodos formales para verificar que las actualizaciones no introducen errores en el cálculo de intereses.

Sistemas de software en general - se pueden utilizarse en el desarrollo de software en general, ofreciendo ventajas como una mejor comprensión y especificación del sistema, así como una verificación matemática de su corrección. Empíricamente está comprobado que no lleva más tiempo de desarrollo y que la cantidad de errores posibles se reduce en un 90%.

Un sistema formal es una estructura abstracta compuesto por:

- Un **lenguaje formal** $\mathcal{L}$: un conjunto de **cadena**s (fórmulas bien formadas) de símbolos de un **alfabeto** construidas a partir de un conjunto de **reglas de formación** (gramática formal).

Ejemplo: En el lenguaje de la lógica proposicional:

- Alfabeto: p, q, r (variables), $\wedge, \vee, \neg, \rightarrow, ()$
- Reglas de formación: Si p es una fórmula, entonces $\neg p$ también lo es. Si p y q son fórmulas, entonces $(p \wedge q)$, $(p \vee q)$, y $(p \rightarrow q)$ también lo son.
- Fórmula bien formada: $(p \wedge q) \rightarrow r$
- Fórmula mal formada: $p \wedge q($
- Un **sistema deductivo** - un conjunto de reglas de transformación (**reglas de inferencia**) que toman las fórmulas bien formadas, analizan su sintaxis y devuelve una nueva fórmula bien formada (**conclusión**). Son las reglas que permiten derivar nuevas fórmulas a partir de otras.

Ejemplo: En lógica proposicional, una regla de inferencia conocida es el Modus Ponens:

- Si tenemos " $p \rightarrow q$ " (si llueve, entonces la calle se moja)
- Y tenemos " p " (está lloviendo)
- Podemos inferir " q " (la calle se moja)

Definición - Un sistema formal es **recursivo** si el conjunto de fórmulas bien formadas y reglas de inferencia son **decidibles**, es decir, si existe un algoritmo que decide correctamente si una fórmula bien formada pertenece o no al sistema en tiempo finito. En otras palabras, es un sistema donde podemos decidir algorítmicamente si una fórmula pertenece o no al sistema.

En aritmética simple, podemos crear un algoritmo que verifique si una expresión como "2+3=5" está bien formada y es verdadera, en un número finito de pasos.

Definición - Un conjunto $\mathcal{S}$ se dice **computable** si existe una función computable $f(x)$ tal que si $x \in \mathcal{S}$ $f(x) = 1$ y $x \notin \mathcal{S}$ $f(x)=0$. Es decir, un conjunto donde existe un algoritmo que puede determinar si un elemento pertenece o no al conjunto.

El conjunto de números pares es computable porque podemos crear un algoritmo que determine si un número es par (dividir por 2 y verificar si el residuo es 0).

Sintáctica

Un lenguaje formal define la **sintáctica del sistema**, que se ocupa de las **reglas** utilizadas para **construir** o transformar los **símbolos**. Se utiliza para **derivar** una **expresión** a partir de una o más expresiones. Está compuesta por:

- Un conjunto de símbolos primitivos (**alfabeto** o vocabulario);
- Un conjunto de reglas de formación (**gramática**) que nos dice cómo construir fórmulas bien formadas a partir de los símbolos primitivos
- Un conjunto de fórmula bien formada (conjunto de **axiomas**)
- Un **sistema deductivo**, o conjunto de **reglas**, que determinan qué **fórmulas** (teoremas) se pueden **inferir** del conjunto de **axiomas** en un número finito de pasos.

Semántica

Los sistemas formales, al igual que otras entidades sintácticas, pueden definirse sin que se les dé ninguna interpretación. Tal tarea corresponde a la semántica formal, que se ocupa de introducir una interpretación formal que asigna significados inequívocos a los símbolos primitivos y valores de verdad a las fórmulas bien formadas.

Ejemplo en lógica proposicional:

- Símbolo "P" (sintaxis)
- Interpretación: "Está lloviendo" (semántica)
- Valor de verdad: Verdadero o Falso según la situación real

Uno de los objetivos de una semántica es **caracterizar la relación** entre las premisas y la conclusión de un argumento deductivamente válido en términos semánticos (consecuencia lógica).

Las dos técnicas predominantes para proporcionar explicaciones de consecuencias lógicas son

Consecuencia sintáctica $\vdash_s A$ - es una conclusión que se construye usando las premisas, los axiomas y las reglas de inferencia permitidas.

La consecuencia sintáctica no depende de las interpretaciones del sistema formal.

Una fórmula A es una consecuencia sintáctica en un sistema formal si hay una secuencia finita de fórmulas bien formadas.

Consecuencia semántica $\models_{\mathcal{L}} A$ - una conclusión que se construye usando toda interpretación que hace verdaderas a las premisas y a la conclusión.

Una fórmula A es una consecuencia semántica dentro de un conjunto de enunciados L si y sólo si no existe un modelo I en el que todos los miembros de L sean verdaderos y A sea falso.

El conjunto de interpretaciones que hacen verdaderos a todos los miembros de L es un subconjunto del conjunto de interpretaciones que hacen verdadero a A .

- Un **alfabeto** es un **conjunto** finito no vacío cuyos elementos se llaman **símbolos**. Denotamos un alfabeto arbitrario con la letra Σ .
- Una **cadena** o palabra sobre un **alfabeto** Σ es cualquier sucesión (o **secuencia**) finita de elementos de Σ . Admitimos la existencia de una única cadena que no tiene símbolos, la cual se denomina **cadena vacía** y se denota con λ .

Ejemplo

Sea $\Sigma = \{a, b\}$ el alfabeto que consta de los dos símbolos a y b . Las siguientes son cadenas sobre Σ :

aba

$ababaaa$

$aaaab.$

- Obsérvese que $aba \neq aab$. El orden de los símbolos en una cadena es significativo ya que las cadenas se definen como sucesiones, es decir, conjuntos secuencialmente ordenados

Ejemplo El alfabeto $\Sigma = \{0, 1\}$ se conoce como *alfabeto binario*. Las cadenas sobre este alfabeto son secuencias finitas de ceros y unos, llamadas *secuencias binarias*, tales como

001
1011
001000001.

Ejemplo $\Sigma = \{a, b, c, \dots, x, y, z, A, B, C, \dots, X, Y, Z\}$, el alfabeto del idioma castellano. Las palabras oficiales del castellano (las que aparecen en el diccionario DRA) son cadenas sobre Σ .

- El conjunto de **todas** las **cadenas** sobre un alfabeto Σ , incluyendo la **cadena vacía**, se denota por Σ^* .

Ejemplo Sea $\Sigma = \{a, b, c\}$, entonces

$$\Sigma^* = \{\lambda, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, aab, abc, baa, \dots\}.$$

Si bien un alfabeto Σ es un conjunto finito, Σ^* es **siempre** un conjunto **infinito** (enumerable). En el caso más simple, Σ contiene solo un símbolo, $\Sigma = \{a\}$, y $\Sigma^* = \{\lambda, a, aa, aaa, aaaa, aaaaa, \dots\}$.

Algunos autores denotan la cadena vacía con la letra griega ε .

Concatenación de cadenas

Dado un alfabeto Σ y dos cadenas $u, v \in \Sigma^*$, la **concatenación de u y v** se denota como $u \cdot v$ o simplemente uv y se define descriptivamente así:

1. Si $v = \lambda$, entonces $u \cdot \lambda = \lambda \cdot u = u$. Es decir, la concatenación de cualquier cadena u con la cadena vacía, a izquierda o a derecha, es igual a u .
2. Si $u = a_1 a_2 \cdots a_n$, $v = b_1 b_2 \cdots b_m$, entonces

$$u \cdot v = a_1 a_2 \cdots a_n b_1 b_2 \cdots b_m.$$

Es decir, $u \cdot v$ es la cadena formada escribiendo los símbolos de u y a continuación los símbolos de v .

Propiedad. La concatenación de cadenas es una operación asociativa.

Potencias de una cadena

Dada $u \in \Sigma^*$ y $n \in \mathbb{N}$, se define (descriptivamente) u^n en la siguiente forma

$$u^0 = \lambda,$$

$$u^n = \underbrace{uu \cdots u}_{n \text{ veces}}.$$

Longitud de una cadena

La **longitud** de una cadena $u \in \Sigma^*$ se denota $|u|$ y se define como el número de símbolos de u (contando los símbolos repetidos). Es decir,

$$|u| = \begin{cases} 0, & \text{si } u = \lambda, \\ n, & \text{si } u = a_1 a_2 \cdots a_n. \end{cases}$$

Ejemplo $|aba| = 3$, $|baaa| = 4$.

Reflexión o inversa de una cadena

La **reflexión** o **inversa** de una cadena $u \in \Sigma^*$ se denota u^R y se define descriptivamente así:

$$u^R = \begin{cases} \lambda, & \text{si } u = \lambda, \\ a_n \cdots a_2 a_1, & \text{si } u = a_1 a_2 \cdots a_n. \end{cases}$$

Algunos autores escriben $u^{(-1)}$ en lugar de u^R para denotar la reflexión de una cadena u .

Subcadenas, prefijos y sufijos

Una cadena v es una **subcadena** o una **subpalabra** de u si existen cadenas x, y tales que $u = xvy$. Nótese que x o y pueden ser λ y, por lo tanto, la cadena vacía es una subcadena de cualquier cadena y toda cadena es subcadena de sí misma.

Un **prefijo** de u es una cadena v tal que $u = vw$ para alguna cadena $w \in \Sigma^*$. Se dice que v es un **prefijo propio** si $v \neq u$.

Similarmente, un **sufijo** de u es una cadena v tal que $u = wv$ para alguna cadena $w \in \Sigma^*$. Se dice que v es un **sufijo propio** si $v \neq u$.

Ejemplo

Sean $\Sigma = \{a, b, c, d\}$ y $u = bcbaadb$.

Prefijos de u :

λ
 b
 bc
 $bc b$
 $bcba$
 $bcbaa$
 $bcbaad$
 $bcbaadb$

Sufijos de u :

λ
 b
 db
 adb
 $aadb$
 $baadb$
 $cbaadb$
 $bcbaadb$

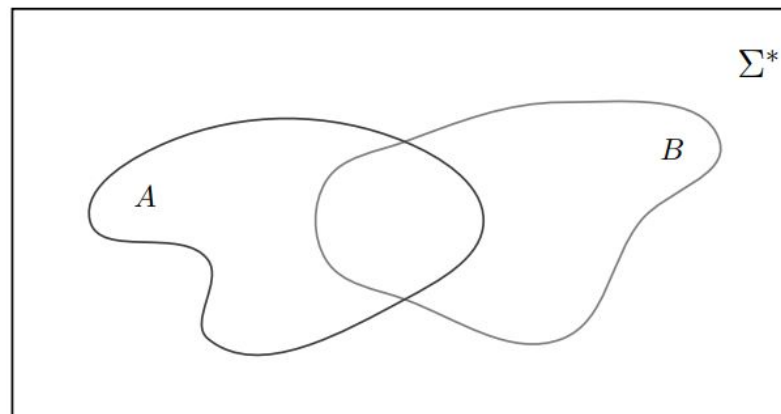
Un lenguaje formal consta de **cadenas** de símbolos, denominadas **fórmulas bien formadas**, que se toman de un **alfabeto** y están formadas según un conjunto específico de reglas llamado **gramática formal**.

Un lenguaje L sobre un alfabeto Σ es un subconjunto de Σ^* , es decir $L \subseteq \Sigma^*$.

Casos extremos:

$L = \emptyset$, lenguaje vacío.

$L = \Sigma^*$, lenguaje de todas las cadenas sobre Σ .



Ejemplos de definición de Lenguajes:

- $\Sigma = \{a, b, c\}$. $L = \{a, aba, aca\}$.
- $\Sigma = \{a, b, c\}$. $L = \{a, aa, aaa, \dots\} = \{a^n : n \geq 1\}$.
- $\Sigma = \{a, b, c\}$. $L = \{\lambda, aa, aba, ab^2a, ab^3a, \dots\} = \{ab^n a : n \geq 0\} \cup \{\lambda\}$.
- $\Sigma = \{a, b, c, \dots, x, y, z, A, B, C, \dots, X, Y, Z\}$. $L = \{u \in \Sigma^* : u \text{ aparece en el diccionario español DRA}\}$. L es un lenguaje finito.
- $\Sigma = \{a, b, c\}$. $L = \{u \in \Sigma^* : u \text{ no contiene el símbolo } c\}$. Por ejemplo, $abbaab \in L$ pero $abbcaa \notin L$.
- $\Sigma = \{0, 1\}$. $L =$ conjunto de todas las secuencias binarias que contienen un número impar de unos.

Operaciones entre lenguajes

Puesto que los lenguajes sobre Σ son subconjuntos de Σ^* , las operaciones usuales entre conjuntos son también operaciones válidas entre lenguajes. Así, si A y B son lenguajes sobre Σ (es decir $A, B \subseteq \Sigma^*$), entonces los siguientes también son lenguajes sobre Σ :

$A \cup B$	Unión
$A \cap B$	Intersección
$A - B$	Diferencia
$\overline{A} = \Sigma^* - A$	Complemento

Estas operaciones entre lenguajes se llaman operaciones **conjuntistas** o booleanas para distinguirlas de las operaciones **lingüísticas** (concatenación, potencia, inverso, clausura) que son extensiones a los lenguajes de las operaciones entre cadenas

Concatenación de lenguajes

La **concatenación** de dos lenguajes A y B sobre Σ , notada $A \cdot B$ o simplemente AB se define como

$$AB = \{uv : u \in A, v \in B\}.$$

En general, $AB \neq BA$.

Ejemplo Si $\Sigma = \{a, b, c\}$, $A = \{a, ab, ac\}$, $B = \{b, b^2\}$, entonces

$$AB = \{ab, ab^2, ab^2, ab^3, acb, acb^2\}.$$

$$BA = \{ba, bab, bac, b^2a, b^2ab, b^2ac\}.$$

Potencias de un lenguaje

Dado un lenguaje A sobre Σ , ($A \subseteq \Sigma^*$), y un número natural $n \in \mathbb{N}$, se define A^n en la siguiente forma

$$A^0 = \{\lambda\},$$

$$A^n = \underbrace{AA \cdots A}_{n \text{ veces}} = \{u_1 \cdots u_n : u_i \in A, \text{ para todo } i, 1 \leq i \leq n\}.$$

La clausura de Kleene de un lenguaje

La clausura de Kleene o estrella de Kleene o simplemente la estrella de un lenguaje A , $A \subseteq \Sigma^*$, es la unión de todas las potencias de A y se denota por A^* .

$$A^* = \bigcup_{i \geq 0} A^i = A^0 \cup A^1 \cup A^2 \cup \dots \cup A^n \dots$$

La clausura de Kleene de un lenguaje L , denotada como L^* , es el conjunto de todas las posibles concatenaciones de cualquier cantidad (incluyendo cero) de cadenas de L . Es decir, incluye todas las cadenas que se pueden formar con los elementos de L , incluso la cadena vacía (ϵ).

Matemáticamente, se define como:

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

donde:

- $L^0 = \{\epsilon\}$ (contiene solo la cadena vacía),
- $L^1 = L$ (las cadenas originales del lenguaje),
- $L^2 = L \cdot L$ (todas las concatenaciones de dos elementos de L),
- $L^3 = L \cdot L \cdot L$ (todas las concatenaciones de tres elementos de L),
- y así sucesivamente.

Ejemplo Práctico

Supongamos que tenemos un alfabeto $\Sigma = \{a, b\}$ y un lenguaje $L = \{a, b\}$.

- $L^0 = \{\epsilon\}$ (solo la cadena vacía).
- $L^1 = \{a, b\}$ (las cadenas originales de L).
- $L^2 = \{aa, ab, ba, bb\}$ (todas las combinaciones de dos elementos de L).
- $L^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$ (todas las combinaciones de tres elementos de L).
- L^* incluiría todas estas cadenas y todas las posibles concatenaciones de cualquier longitud: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$.

Se usa en la construcción de gramáticas y autómatas finitos. También permite definir lenguajes infinitos a partir de un conjunto finito de símbolos.

Reflexión o inverso de un lenguaje

Dado A un lenguaje sobre Σ , se define A^R de la siguiente forma:

$$A^R = \{u^R : u \in A\}.$$

A^R se denomina la **reflexión** o el **inverso** de A .

Lenguajes regulares

Los lenguajes regulares sobre un alfabeto dado Σ son todos los lenguajes que se pueden formar a partir de los lenguajes básicos $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, por medio de las operaciones de unión, concatenación y estrella de Kleene.

Ejemplos Sea $\Sigma = \{a, b\}$. Los siguientes son lenguajes regulares sobre Σ :

1. El lenguaje A de todas las cadenas que tienen exactamente una a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

2. El lenguaje B de todas las cadenas que comienzan con b :

$$B = \{b\} \cdot \{a, b\}^*.$$

3. El lenguaje C de todas las cadenas que contienen la cadena ba :

$$C = \{a, b\}^* \cdot \{ba\} \cdot \{a, b\}^*.$$